

Is the system hackable

Short answer: **Hacking this system is extremely difficult — practically impossible with today's technology — *if* keys are kept safe.**

Let me explain it clearly, realistically, and in a way you can present.

Can Someone Hack This System? (Honest, Practical Explanation)

Your system uses **three layers of modern, industry-grade security**:

Layer 1 — AES-256-GCM Encryption (for data)

Military-grade encryption

AES-256 is currently considered **unbreakable**.

Brute-forcing AES-256 would take:

❌ Not just years

❌ Not centuries

→ It would take *longer than the age of the universe*

Even supercomputers cannot crack AES-256.

So, the data stored in IPFS cannot be decrypted by anyone.

Layer 2 — Elliptic Curve Cryptography (ECC, secp256k1)

This is the same cryptography used by:

- Bitcoin
- Ethereum
- All modern blockchain wallets

Breaking ECC (secp256k1) means:

- You can steal Bitcoin
- Hack Ethereum
- Break the internet

This is **also considered practically impossible** with current computers.

Layer 3 — Umbral Proxy Re-Encryption (PRE)

This prevents:

- ✓ Owner's private key from being exposed
- ✓ Proxy from seeing plaintext
- ✓ Recipients from impersonating owner
- ✓ Anyone from re-encrypting without permission

Even if IPFS leaks all files:

- They are encrypted
 - Proxy cannot decrypt
 - Only allowed recipients can decrypt
-

So can it be hacked?

Let's break it down realistically:

1. Can someone decrypt the IPFS data?

NO.

Because the IoT data is protected with **AES-256-GCM**.

2. Can the proxy see the plaintext?

NO.

Proxy sees only:

- ciphertext
- capsules
- kfrag fragments

It performs math, not decryption.

3. Can an attacker fake access to the data?

NO.

They would need:

- Owner private key → impossible
 - Valid kfrags → impossible
 - Recipient private key → impossible
-

4. Can someone hack PRE? (Umbral)

Extremely unlikely.

No real-world breaks discovered.

Based on peer-reviewed cryptography.

Used in NuCypher + Threshold Network.

5. Can blockchain part be hacked?

Not if written correctly, because:

- Smart contract only stores metadata (no secrets)
 - All critical security happens off-chain
-

Where attacks *can* happen (realistic risks)

THIS PART IS IMPORTANT FOR YOUR PRESENTATION.

No system is 100% safe if humans make mistakes.

Here are the real-world risks:

Risk 1: Private Key Leaks

If the **owner's or recipient's private key** is exposed, the system is compromised.

This is the biggest risk.

Solution:

- Store keys securely
 - Use encrypted key files
 - Never share private keys
-

Risk 2: Compromised machine

If a hacker gets into the client machine running the Python code:

- They can steal keys
- They can intercept decrypted data

This is a device security problem, not encryption problem.

Risk 3: Poor implementation

Security depends on correct implementation.

But your implementation uses:

- pyUmbra1 (well-tested library)
- AES-256-GCM
- ECC secp256k1

So crypto is not the weak part.

Risk 4: Fake RPC or IPFS node

If attacker controls the network and changes RPC or IPFS endpoints:

- They could block access
- But they cannot decrypt data

★ FINAL ANSWER**

“The encryption we use—AES-256 + Elliptic Curve Cryptography + Proxy Re-Encryption—is the same level of security used in blockchain networks and military systems.

As long as private keys are safe, the system is practically unhackable.

The only real risks come from poor key storage or compromised devices, not from breaking the encryption itself.”

🔒 Question: Can a hacker “decrypt twice” or break the system during the 2-step decryption process?

No — a hacker cannot decrypt at either step of the process.

Here's why 🙋

🧩 How the PRE Decryption Works (Very Simple)

There are **two independent** decryptions:

Step 1 → Proxy Re-Encryption (Transform step, NOT decryption)

- Proxy does **not** decrypt.
- Proxy only transforms the “capsule”.
- Proxy never gets the symmetric key.
- Proxy sees ciphertext, never plaintext.

This step is **not real decryption**.

It's more like converting a lock so another person's key can open it.

A hacker cannot exploit this step because:

- There is no plaintext.
 - No keys are exposed.
 - Only math transformation happens.
-

Step 2 → Recipient Decryption (Actual decryption)

Recipient decrypts using **their own private key**.

A hacker **cannot** decrypt at this step because:

- They don't have the recipient's private key.
 - They don't have the owner's private key.
 - They don't have the AES-256 key.
 - They cannot forge valid cfrags.
 - The AES-256-GCM ciphertext cannot be brute-forced.
-

So what about “decrypting twice”?

There is **no point** in the process where a hacker can:

- ✗ decrypt at proxy
- ✗ decrypt at owner
- ✗ decrypt at IPFS
- ✗ decrypt at access grant
- ✗ decrypt the capsule
- ✗ decrypt during re-encryption
- ✗ decrypt by replaying the process
- ✗ decrypt even if they intercept everything

Why?

- 👉 Because the ciphertext **never becomes plaintext at any intermediate point**
 - 👉 Only the final authorized recipient sees plaintext
 - 👉 The proxy never sees plaintext
 - 👉 AES-256-GCM & ECC make decryption mathematically infeasible
-

What if the hacker tries BOTH decryptions?

There is **no second decryption** to exploit.

The process is:

- 1 Owner encrypts → produces ciphertext + capsule
- 2 Proxy transforms capsule → still ciphertext
- 3 Recipient decrypts → gets plaintext

A hacker:

- cannot decrypt step 1
- cannot decrypt step 2
- cannot bypass step 2
- cannot decrypt even if they repeat the process 100 times

Because they don't have the recipient's private key.

 **Even if a hacker gets ALL of these things, they STILL cannot decrypt:**

- The encrypted file from IPFS
- The capsule
- The kfrags
- The cfrags
- The access grant metadata
- The hash
- The encrypted re-encrypted data

None of this gives them the AES key or recipient private key.

 **Short, presentation-friendly answer**

“No. A hacker cannot decrypt at any step.

The proxy never decrypts — it only transforms encrypted data.

Only the recipient can perform the *one and only* decryption using their private key.”

★ **Ultra-simple version (for non-technical audience)**

“There are not two decryptions.

The proxy only reshapes the encrypted data; it never sees the actual content.

Only the final recipient does the actual decryption.

So there’s no chance for a hacker to decrypt it anywhere else.”